

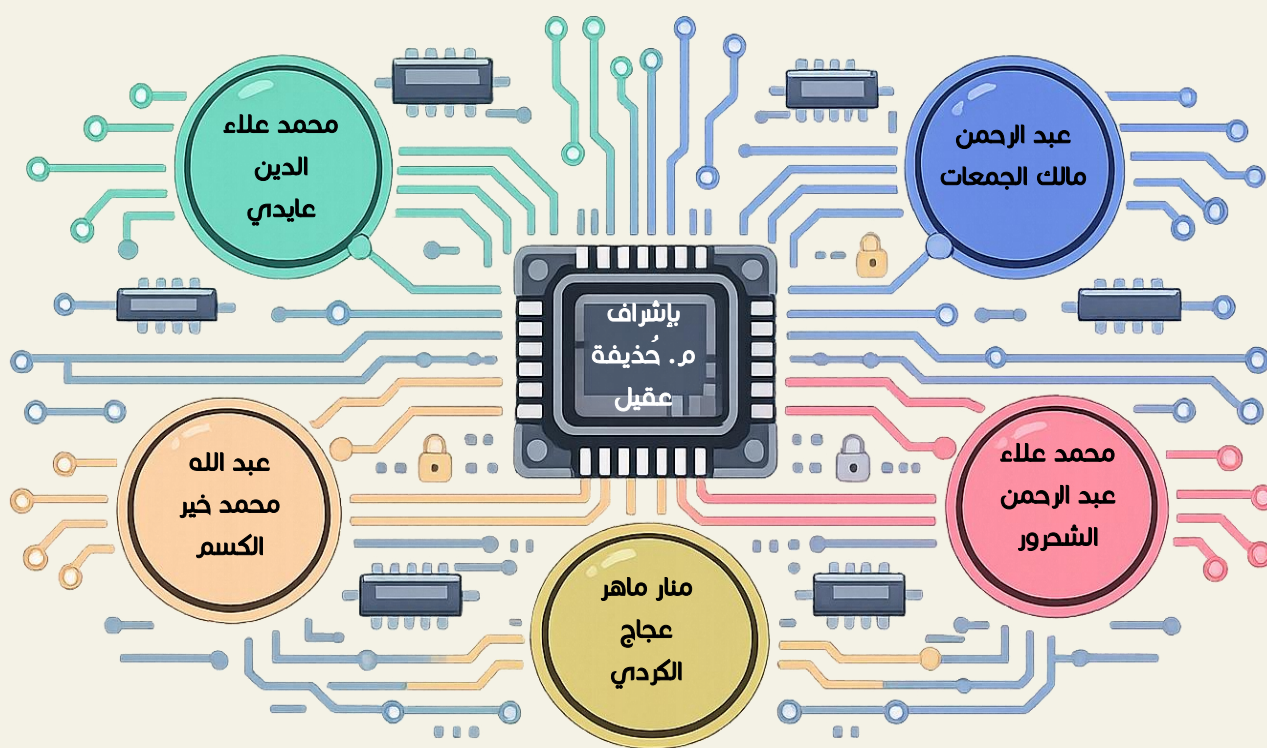


مشروع البرمجة التفرعية

تطبيق التجارة الإلكترونية

تاريخ 18/05/2026

أعضاء الفريق



المحتويات

5	الطلب الأول: حماية البيانات المشتركة من التضارب Concurrent Access & Data Integrity
5	السيناريو الأول: إضافة منتج للسلة من جهازين
6	قبل اضافة القيد:
6	بعد اضافة القيد:
6	السيناريو الثاني: تحديث كمية المنتج في السلة ((Lost Update)
6	حل المشكلة:
7	(الكود كامل قبل وبعد ضمن cart service)
7	النتيجة قبل الحل:
8	بعد الحل:
8	السيناريو الثالث: إتمام الطلب من جهازين ((Double Checkout)
8	الكود كامل ضمن order service
9	قبل الحل:
9	السيناريو الرابع: زبونان يشتريان آخر وحدة ((Overselling)
10	الكود كامل ضمن الـ OrderService
10	قبل الحل:
11	بعد الحل:
11	السيناريو الخامس: شراء وتحديث مخزون بالتزامن ((Hidden Inventory Update)
11	الكود كامل ضمن الـ Inventory Service
12	قبل الحل:
12	بعد الحل:
13	الطلب الثاني Rate-Limiter
15	استخدام Redis لتتبع سجلات الـ Rate-Limiter:
15	أين استخدمنا الـ Rate Limiters ولماذا:
16	الاختبار باستخدام أداة K6
17	النتائج:
17	قبل وصول المستخدم للحد الأقصى من الطلبات المتاحة له خلال مدة معينة:
17	بعد وصول المستخدم إلى الحد المخصص له خلال مدة معينة:
	الطلب الثالث: المعالجة غير المتزامنة: Asynchronous Queues نقل المهام التي لا يحتاج المستخدم لانتظارها (مثل
18	إصدار الفواتير) خارج المسار الرئيسي للطلب
18	عرض المشكلة:

18	من نظرة معمارية :
19	1. Laravel Queue System
19	2. Jobs Layer
19	3. Dispatch Mechanism
19	4. إعدادات إعادة المحاولة عند الفشل :
20	5 - فحص وجود الملف مسبقاً وتحديث قاعدة البيانات (Idempotency & Path Registration)
21	تدفق النظام (System Flow)
21	أهم أجزاء الكود (Key Code Snippets Explained)
21	عملية ال dispatch لل Job :
22	تحميل البيانات داخل ال Job
	Error! Bookmark not defined. : النتائج
24	الطلب الرابع: معالجة البيانات على دفعات Batch Processing in Parallel Environments
24	آلية العمل
24	لماذا استخدمنا Batch Processing
24	مقارنة المعالجة التقليدية مع المعالجة على دفعات
25	قياس الأداء واستهلاك الذاكرة
25	الإحصائيات داخل التقرير
25	النتائج
26	والنتائج التفصيلية:
28	الطلب الخامس: دراسة وتنفيذ نظام توزيع الأحمال (Load Balancing)
28	المقدمة والهدف من النظام
28	الدراسة النظرية: استراتيجيات توزيع الأحمال
28	Round Robin
28	Weighted Round Robin
29	Least Connections
29	Weighted Least Connections
30	IP Hash
30	التنفيذ العملي: المعمارية التقنية
31	شرح الملفات التي تم اعدادها
31	Dockerfile
32	docker-compose.yml

{

33.....	إعدادات NGINX الخاصة بكل تطبيق
33.....	إعداد PHP-FPM
34.....	إعداد Load Balancer باستخدام NGINX
34.....	النتيجة العملية
35.....	لوحة التحكم والمراقبة (Dashboard)
36.....	النتائج عند وجود Node واحدة فقط (أي وجود ال load balancer وعدمه واحد)
36.....	النتائج عند تشغيل 3 عقد (3 سيرفرات) واستخدام load balancer لتوزيع الحمل:

المتطلبات الغير وظيفية:

الطلب الأول:

حماية البيانات المشتركة من التضارب Concurrent Access & Data Integrity

#	السيناريو	المورد المشترك	المشكلة النت
1	إضافة منتج للسلة من جهازين	جدول cart_items	Duplicate Insert
2	تحديث كمية منتج في السلة	صف cart_items	Lost Update
3	إتمام الطلب من جهازين	الرصيد + المخزون	Double Checkout
4	شراء زبونين لآخر قطعة من نفس المنتج	صف المخزون	Overselling
5	إداري يحدد المخزون لمنتج وزبون يقوم بشراء المنتج	صف المخزون	Hidden Inventory Update

لحل مشاكل ال race condition ضمن لارافيل يوجد لدينا 4 حلول:

1. قفل الجزء من الكود الذي يحدث فيه التضارب
2. قفل السجلات في قاعدة البيانات باستخدام lockForUpdate (قفل تشاؤمي Pessimistic Lock)
3. استخدام sharedLock ممكن إذا أردنا منع الآخرين من الكتابة والسماح لهم بقراءة السجلات، في حالة التضارب الثريدين يريدان القيام بعملية كتابة فلا ينفع استخدامه.
4. استخدام القفل المتفائل لا يحل المشكلة، لأنه يعتمد على version مع كل اصدار، وبفرض استعلام الثريدين عن الاصدار قبل التحديث بنفس اللحظة سيقومان بالتحديث سويا، ما لم نستخدم القفل التشاؤمي معه.

السيناريو الأول: إضافة منتج للسلة من جهازين

المشكلة: عندما يضغط المستخدم على زر الإضافة من جهازين في آن واحد، تقوم كلتا العمليتين بالتحقق من عدم وجود المنتج في السلة، فتتم اضافة المنتج للسلة مرتين، حل المشكلة عن طريق اضافة قيد على جدول cart_items بحيث يكون المنتج فريد ضمن السلة الواحدة.

قبل اضافة القيد:

```
— Request 1 (Device A) ————— source=console
INFO[0000] HTTP Status : 200 source=console
INFO[0000] Success : true source=console
INFO[0000] Message : Product added to cart source=console
INFO[0000]
— Request 2 (Device B) ————— source=console
INFO[0000] HTTP Status : 200 source=console
INFO[0000] Success : true source=console
INFO[0000] Message : Product added to cart source=console
INFO[0000] ===== source=console
```

بعد اضافة القيد:

```
— Request 1 (Device A) ————— source=console
INFO[0000] HTTP Status : 422 source=console
INFO[0000] Success : false source=console
INFO[0000] Message : Product already in cart source=console
INFO[0000]
— Request 2 (Device B) ————— source=console
INFO[0000] HTTP Status : 200 source=console
INFO[0000] Success : true source=console
INFO[0000] Message : Product added to cart source=console
INFO[0000] ===== source=console
```

السيناريو الثاني: تحديث كمية المنتج في السلة ((Lost Update

يدخل المستخدم جهازين ويغيّر كمية نفس المنتج من الجهازين في آن واحد. تقرأ كلتا العمليتين القيمة الحالية (مثلاً 5)، ثم تكتب كل منهما قيمتها الجديدة (3 و8)، سيهمل أحد التغيرين ويبقى الآخر دون اشعار المستخدم.

حل المشكلة:

استخدام ال Pessimistic Lock غير مفيد، لأننا نقوم بعمل assign لقيمة بالكود وليس increment - decrement، لو كان الكود يقوم بعمل increment - decrement سيكون خيار مناسب.

سنستخدم Atomic Lock، نقوم بقفل مقطع تحديث المنتج ضمن السلة بحيث لا يمكن الا لثريد واحد ان يحدث، وبهال اتي طلب آخر ولم يتمكن من امتلاك القفل نرفض عملية التحديث.

(الكود كامل قبل وبعد ضمن cart service)

```
{  
    $cart = $user->cart;  
    $lock = Cache::lock( name: "update_cart:$cart->id:$productId");  
    try {  
        if (!$lock->get())  
            return ['success' => false, 'message' => 'Quantity updated From Another DeviceL'];  
        // Update Quantity Logic  
        return ['success' => true, 'message' => 'Quantity updated successfully'];  
    } finally {  
        $lock->release();  
    }  
}
```

النتيجة قبل الحل:

```
— Request 1 ————— source=console  
INFO[0000] Wanted qty : 8 source=console  
INFO[0000] HTTP Status : 200 source=console  
INFO[0000] Success : true source=console  
INFO[0000] Message : Quantity updated successfully source=console  
INFO[0000]  
— Request 2 ————— source=console  
INFO[0000] Wanted qty : 9 source=console  
INFO[0000] HTTP Status : 200 source=console  
INFO[0000] Success : true source=console  
INFO[0000] Message : Quantity updated successfully source=console  
INFO[0000]  
— Final State ————— source=console  
INFO[0000] Quantity BEFORE race : 5 source=console  
INFO[0000] Request 1 wanted : 8 source=console  
INFO[0000] Request 2 wanted : 9 source=console  
INFO[0000] Quantity AFTER race : 9 source=console  
INFO[0000] ===== source=console
```

بعد الحل:

```
— Request 1 — source=console
INFO[0000] Wanted qty : 1 source=console
INFO[0000] HTTP Status : 422 source=console
INFO[0000] Success : false source=console
INFO[0000] Message : Quantity updated From Another DeviceL source=console
INFO[0000]
— Request 2 — source=console
INFO[0000] Wanted qty : 1 source=console
INFO[0000] HTTP Status : 200 source=console
INFO[0000] Success : true source=console
INFO[0000] Message : Quantity updated successfully source=console
INFO[0000]
— Final State — source=console
INFO[0000] Quantity BEFORE race : 5 source=console
INFO[0000] Request 1 wanted : 1 source=console
INFO[0000] Request 2 wanted : 1 source=console
INFO[0000] Quantity AFTER race : 1 source=console
INFO[0000] ===== source=console
```

السيناريو الثالث: إتمام الطلب من جهازين (Double Checkout))

المشكلة: يضغط المستخدم زر الشراء مرتين متتاليتين (أو تُرسل الطلبات من جهازين). فيُنشئ النظام طلبين منفصلين ويخصم الرصيد مرة أو مرتين.

آلية الحل: باستخدام Atomic Lock مرتبط بمعرف المستخدم يمنع الدخول المتزامن لنفس الحساب الى كود ال checkout. يُحتفظ بالقفل طوال العملية كاملة بما فيها إنشاء الطلب وخصم الرصيد وترفض اي عملية checkout لنفس اليوزر في هذه الأثناء.

الكود كامل ضمن order service

```
{
    $user = Auth::user();
    $perUserLock = Cache::lock( name: "checkout:user:{$user->id}");// preventing the same user to checkout
    if (!$perUserLock->get()) {
        return ['success' => false, 'message' => 'Checkout in progress'];
    }
    try {
        $res = DB::transaction(function () use ($data, $perUserLock, $user, $inventoryLock) {
            // Create the order.
            $perUserLock->release();
            // return success result
        });
        return $res;
    } finally {
        $perUserLock->release();
    }
}
```


قبل الحل:

```
— Request 1 — source=console
INFO[0001] HTTP Status : 201 source=console
INFO[0001] Success      : true source=console
INFO[0001] Message      : Payment completed successfully source=console
INFO[0001] Order ID     : 2 source=console
INFO[0001] Wallet after: $689.06 source=console
INFO[0001]
— Request 2 — source=console
INFO[0001] HTTP Status : 201 source=console
INFO[0001] Success      : true source=console
INFO[0001] Message      : Payment completed successfully source=console
INFO[0001] Order ID     : 1 source=console
INFO[0001] Wallet after: $689.06 source=console
INFO[0001]
===== source=console
```

بعد الحل:

```
— Request 1 — source=console
INFO[0000] HTTP Status : 422 source=console
INFO[0000] Success      : false source=console
INFO[0000] Message      : Checkout in progress source=console
INFO[0000]
— Request 2 — source=console
INFO[0000] HTTP Status : 201 source=console
INFO[0000] Success      : true source=console
INFO[0000] Message      : Payment completed successfully source=console
INFO[0000] Order ID     : 1 source=console
INFO[0000] Wallet after: $844.03 source=console
INFO[0000] ===== source=console
```

السيناريو الرابع: زبونان يشتريان آخر وحدة (Overselling)

المشكلة: يوجد منتج كميته 1 فقط. يحاول زبونان شراؤه في نفس اللحظة. يقرأ كلاهما الكمية = 1، يجتازان فحص المخزون، ويكملان الشراء. النتيجة: مبيعاتان لمنتج لا يوجد منه سوى وحدة واحدة (Overselling).

آلية الحل: استخدام lockForUpdate داخل DB::transaction يضمن أن العملية التي تحصل على القفل أولاً تقرأ الكمية الحقيقية وتعدلها، بينما تنتظر العملية الثانية وتقرأ الكمية المحدثة (0) فترفض الطلب.

```

$inventories = Inventory::whereIn( column: 'product_id', $productIds)
    ->orderBy( column: 'product_id')
    ->lockForUpdate()
    ->get()
    ->keyBy( keyBy: 'product_id');

```

قبل الحل:

```

INFO[0001]
— VU 1 (buyer1@example.com) ————— source=console
INFO[0001] HTTP Status : 201 source=console
INFO[0001] Success : true source=console
INFO[0001] Message : Payment completed successfully source=console
INFO[0001] Order ID : 1 source=console
INFO[0001]
— VU 2 (buyer2@example.com) ————— source=console
INFO[0001] HTTP Status : 201 source=console
INFO[0001] Success : true source=console
INFO[0001] Message : Payment completed successfully source=console
INFO[0001] Order ID : 2 source=console
INFO[0001]
— Final State ————— source=console
INFO[0001] Quantity BEFORE race : 1 source=console
INFO[0001] Quantity AFTER race : 0 source=console
INFO[0001] ===== source=console

```

بعد الحل:

```
===== source=console
INFO[0000]
— VU 1 (buyer1@example.com) — source=console
INFO[0000] Body : [object Object] source=console
INFO[0000] HTTP Status : 201 source=console
INFO[0000] Success : true source=console
INFO[0000] Message : Payment completed successfully source=console
INFO[0000] Order ID : 1 source=console
INFO[0000]
— VU 2 (buyer2@example.com) — source=console
INFO[0000] Body : [object Object] source=console
INFO[0000] HTTP Status : 422 source=console
INFO[0000] Success : false source=console
INFO[0000] Message : Some products are out of stock source=console
INFO[0000]
— Final State — source=console
INFO[0000] Quantity BEFORE race : 1 source=console
INFO[0000] Quantity AFTER race : 0 source=console
INFO[0000]
===== source=console
```

السيناريو الخامس: شراء وتحديث مخزون بالتزامن (Hidden Inventory Update))

المشكلة: المخزون يحتوي 40 قطعة، المسؤول يرغب بإضافة 20 قطعة للمخزون، في نفس اللحظة يشتري زبون 10 قطع. اعتمادا على من يصل أولا قد تصبح قيمة المخزون اما 50 (صحيحة) او 30 (نتيجة لتأخر كتابة عملية الزبون) او 60 (نتيجة لتأخر كتابة عملية المسؤول).

آلية الحل: استخدام القفل التبادلي

الكود كامل ضمن ال Inventory Service

```
return DB::transaction(function () use ($productId, $quantity) {

    $inventory = Inventory::where( column: 'product_id', $productId)
        ->lockForUpdate()
        ->first();
```

قبل الحل:

```
-----
-- VU2 ADMIN (set qty=60) ----- source=console
INFO[0001] HTTP Status : 200 source=console
INFO[0001] Success : true source=console
INFO[0001] Message : Inventory updated successfully source=console
INFO[0001]
-- VU1 BUYER (checkout 10 units) ----- source=console
INFO[0001] HTTP Status : 201 source=console
INFO[0001] Success : true source=console
INFO[0001] Message : Payment completed successfully source=console
INFO[0001] Wallet after: $200.10 source=console
INFO[0001] Order ID : 1 source=console
INFO[0001]
-- Final State ----- source=console
INFO[0001] Qty BEFORE race : 40 source=console
INFO[0001] Buyer purchased : 10 units source=console
INFO[0001] Admin tried to increment : 20 source=console
INFO[0001] Expected safe final qty: 50 source=console
INFO[0001] Actual final qty : 30 source=console
INFO[0001] ===== source=console
```

بعد الحل:

```
-----
-- VU2 ADMIN (set qty=60) ----- source=console
INFO[0000] HTTP Status : 200 source=console
INFO[0000] Success : true source=console
INFO[0000] Message : Inventory updated successfully source=console
INFO[0000]
-- VU1 BUYER (checkout 10 units) ----- source=console
INFO[0000] HTTP Status : 201 source=console
INFO[0000] Success : true source=console
INFO[0000] Message : Payment completed successfully source=console
INFO[0000] Wallet after: $200.10 source=console
INFO[0000] Order ID : 1 source=console
INFO[0000]
-- Final State ----- source=console
INFO[0000] Qty BEFORE race : 40 source=console
INFO[0000] Buyer purchased : 10 units source=console
INFO[0000] Admin tried to increment : +20 source=console
INFO[0000] Expected safe final qty : 50 source=console
INFO[0000] Actual final qty : 50 source=console
INFO[0000] ===== source=console
```

الطلب الثاني

إدارة الموارد الحاسوبية (Resource Management & Capacity Control)

في هذا القسم سوف نتحدث عن أمران:

إدارة الموارد الحاسوبية وال Rate limiting لحماية الموارد الحاسوبية

وفي قسم إدارة الموارد الحاسوبية، قمنا بإجراء جميع التجارب على نظام التشغيل Windows، وقمنا باستخدام Nginx كمخدم و واستخدمنا FastCGI كبديل عن PHP-fpm

لكن، في طلب ال Load Balancer، قمنا بإعادة التجربة كاملة على نظام Linux وباستخدام PHP-FPM مماثلة تماما للموجودة على المخدمات الفعلية في السوق، وكانت التجربة أفضل بكثير وأساسا ولدينا تحكم أعلى بالموارد

إدارة الموارد الحاسوبية

بداية الوضع كان سيئ كثير وكل طلب عم يأخذ وسطيا 30 ثانية ليتعالج

وكان عدد الطلبات يلي بعالجاها بالثانية الوحدة وسطيا هنن بس

3 req/s وكل طلب يأخذ ما يقارب ال 28 ثانية لأنه ينتظر في ال Queue

انا فتحت ال task manager لشوف اناو شو المشكلة

App history	CLI	5.5%	12.2 MB	0.1 MB/s
Startup apps	System	1.3%	0.1 MB	0.3 MB/s

لاحظت اناو php عم تستهلك من المعالج 5 بالمية فقط

فانا استغربت قلت معناها 95 بالمية هنن موارد مهدورة ما عم تستغلهم php

وهون بليشت الرحلة

استخدام php Fast CGI

وهي عبارة عن php workers يقوم كل عامل بمعالجة الطلبات بشكل منفصل عن البقية، أي نصبح قادرين على معالجة الطلبات بالتوازي، وقد قمنا بإنشاء 20 عامل لمعالجة الطلبات على التوازي وكل عامل يعالج 1000 طلب قبل أن يتم إيقاف هذه العامل وخلق عامل جديد

استخدام OPcache

وهي خدمة من php للقيام ب Cache لملفات ال php

تشغيل وضع ال Release Mode

من أهم الأمور التي قد يتغاضى عنها المبرمج هو عدم الانتباه إلى أن التطبيق يعمل في وضع debug، مما يؤدي إلى بطيء في الأداء

Keepalive requests

وهذه القيمة وضعتها 1000 أي يبقى كل اتصال حي ويعالج 1000 طلب قبل أن يتم إغلاق هذا الاتصال، أو يتم إغلاقه بعد أن يبقى الاتصال غير فعال لمدة 65 ثانية
وهذه تفيد في توفير الوقت اللازم لإنشاء وإغلاق الاتصالات في كل مرة يتم فيها إرسال طلب للسيرفر

Nginx Workers

وهم عمال من طرف ال Nginx وهم المسؤولين عن إدارة ال Connections واستقبال طلبات ال https التي تصل للسيرفر وهذه القيمة وضعناها auto فيقرر ال Nginx القيمة الأفضل حسب الموارد المتاحة

Nginx Worker Connections

وهذه القيمة وضعتها 4096 أي أن كل عامل سوف يقوم بخلق 4000 اتصال، فيصبح عدد الاتصالات بين السيرفر والمستخدمين هو عدد العمال من nginx * عدد الاتصالات لكل عامل

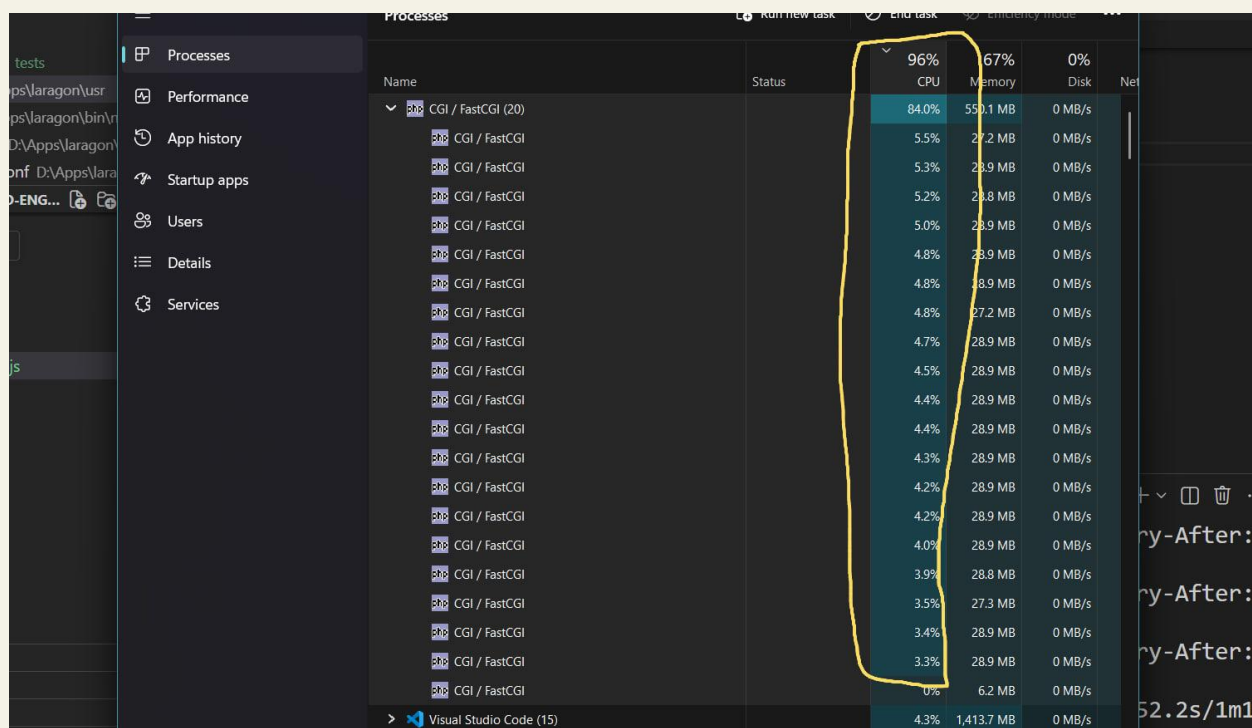
Upstream Connections

وهذه هي عدد الاتصالات بين ال Nginx وال php application ووضعت هذه القيمة 32 مع إبقاء هذه الاتصالات حية بنفس الشروط السابقة.

النتائج:

وهذه النتائج على نظام ويندوز (قمنا بتنفيذها على لينوكس ب setup أعقد ولكن سنكلم عنه في قسم ال Load Balancing)

زاد عدد الطلبات المعالجة من 3 req / s إلى 167 request / s، وقل زمن الاستجابة للطلب الواحد من 28 ثانية ووصل إلى 119 ميلي ثانية
وزادت استهلاك قوة المعالج من 8 بالمائة حتى وصلت ل 96% بالمية.



Name	Status	CPU	Memory	Disk	Net
CGI / FastCGI (20)		96%	67%	0%	
CGI / FastCGI		84.0%	550.1 MB	0 MB/s	
CGI / FastCGI		5.5%	27.2 MB	0 MB/s	
CGI / FastCGI		5.3%	28.9 MB	0 MB/s	
CGI / FastCGI		5.2%	28.8 MB	0 MB/s	
CGI / FastCGI		5.0%	28.9 MB	0 MB/s	
CGI / FastCGI		4.8%	28.9 MB	0 MB/s	
CGI / FastCGI		4.8%	28.9 MB	0 MB/s	
CGI / FastCGI		4.8%	27.2 MB	0 MB/s	
CGI / FastCGI		4.7%	28.9 MB	0 MB/s	
CGI / FastCGI		4.5%	28.9 MB	0 MB/s	
CGI / FastCGI		4.4%	28.9 MB	0 MB/s	
CGI / FastCGI		4.4%	28.9 MB	0 MB/s	
CGI / FastCGI		4.3%	28.9 MB	0 MB/s	
CGI / FastCGI		4.2%	28.9 MB	0 MB/s	
CGI / FastCGI		4.2%	28.9 MB	0 MB/s	
CGI / FastCGI		4.0%	28.9 MB	0 MB/s	
CGI / FastCGI		3.9%	28.8 MB	0 MB/s	
CGI / FastCGI		3.5%	27.3 MB	0 MB/s	
CGI / FastCGI		3.4%	28.9 MB	0 MB/s	
CGI / FastCGI		3.3%	28.9 MB	0 MB/s	
CGI / FastCGI		0%	6.2 MB	0 MB/s	
Visual Studio Code (15)		4.3%	1,413.7 MB	0 MB/s	

Rate-Limiter

في هذا القسم سوف نقوم بحماية موارد الجهاز وال End-Points الخاصة بالسيرفر من الاستخدام المفرط، حيث أن هذه الطريقة تحدّ من عدد الطلبات التي يمكن للمستخدم من ارسالها خلال فترة زمنية معينة، وبهذا نحمي الخادم من هجمات ال DDoS التي تحاول استهلاك موارد الجهاز، أو هجمات ال Brute Force التي تستخدم لمعرفة كلمات المرور، أو أي إساءة استخدام لموارد الجهاز

ولتتبع المستخدم ومعرفة كل شخص يقوم باستخدام ال End-points الخاصة بالسيرفر، نقوم باستخدام ال ID الخاص بالمستخدم ان كان قد سجل دخول في النظام، أو IP المستخدم ان كان دخوله للسيرفر بشكل زائر دون وجود حساب خاص به.

استخدام Redis لتتبع سجلات ال Rate-Limiter:

ان Laravel يقوم بدعم ال Rate Limiting بشكل افتراضي عن طريق ال Cache، لكنني استخدمت Redis للأسباب التالية:

- Performance: ان ال Redis تستخدم الذاكرة المؤقتة للخادم مما يعطينا أداء وسرعة أكبر من قواعد البيانات التقليدية
- الدقة: يتعامل مع الطلبات المتزامنة ويتجنب ال race condition بفضل عمليات ال atomic
- ال TTL: يدعم انتهاء الصلاحية التلقائي للمفاتيح

أين استخدمنا ال Rate Limiters ولماذا:

الحد الأقصى	الوسيلة المتبعة لتتبع المستخدم	سبب استخدام ال rate limiter	Limiters ال
120/min	User ID if not then IP	حماية ال Endpoints العامة من إساءة الاستخدام	Public-api
120/min	User ID	حماية ال Endpoints العامة من إساءة الاستخدام	Authenticated-api
5/min	Email + IP	منع هجمات ال Brute Force لمعرفة كلمة المرور	Login
3/min	IP	منع إنشاء حسابات وهمية بشكل اوتوماتيكي، وحماية موارد الجهاز	Register
60/min	User ID	حماية ال Endpoint العامة من إساءة الاستخدام	Cart

حماية ال Endpoint العامة من إساءة الاستخدام	User ID	5/min	Checkout
حماية ال Endpoint العامة من إساءة الاستخدام	User ID	3/min	Wallet
منع إساءة استخدام موارد الجهاز	User ID	20/min	Inventory-Update
منع إساءة استخدام موارد الجهاز	User ID	10/min	Admin-actions

الاختبار باستخدام أداة K6

تم كتابة ملف Rate-Limit-Tests.js باستخدام أداة K6 للاختبار أن ال Rate Limiter يعمل وفق الشكل المطلوب وهذا عن طريق ارسال عدد كبير من الطلبات لنفس المستخدم خلال مدة زمنية جدا قصيرة

$\{$ $\{$ $\{$ $\{$ $\{$ $\{$ $\{$

وفي هذه الصورة نلاحظ أن المستخدم قد وصل إلى الحد الأقصى له، ويقوم السيرفر بإرجاع 429 ويعني: Too Many Requests، ونلاحظ أن ال remaining يساوي ال 0، وال Retry-After هو 40 أي متبقي 40 ثانية حتى يتمكن المستخدم من جديد من استخدام هذه ال endpoint، **ملاحظة:** نلاحظ أن ال k6 تقوم بإرسال العديد من الطلبات خلال نفس الثانية، لهذا نرى أن عداد ال Retry-After يساوي ال 40 في جميع الطلبات لأنها جميعاً تم إرسالها في نفس الثانية

الطلب الثالث:

المعالجة غير المتزامنة: Asynchronous Queues نقل المهام التي لا يحتاج المستخدم للانتظارها (مثل اصدار الفواتير) خارج المسار الرئيسي للطلب

عرض المشكلة:

في بعض العمليات التي نجريها في مشروعنا نجد أنه يتم تنفيذ مجموعة عمليات ضمن نفس ال request مما يعني أن المستخدم سينتظر اكتمال جميع المهام المرتبطة بهذا ال request قبل الحصول على ال response

كمثال على هذه المشكلة لدينا عملية checkout من ضمن مراحلها إنشاء فاتورة لما اشتراه الزبون وتصديرها كملف بصيغة PDF وهذه العملية تعد من العمليات الغير حرجية الثقيلة من ناحية المعالجة ونقصد بالدرجة أي انها تتبع عملية checkout وفشلها لا يؤثر بشكل قوي بعملية checkout عدا أنها تسبب ما يلي عند تنفيذها بشكل متزامن مع عملية checkout:

- ارتفاع زمن استجابة API
- تأثر تجربة المستخدم بالسلب
- يصبح النظام أقل قابلية للتوسع تحت الضغط

وهذه هي الامور الثلاث التي سنتوصل في نهاية الطلب الى مقارنتها وذكر كيف تم تحسينها

بدلاً من تنفيذ جميع العمليات المرتبطة بعملية ال Checkout بشكل متزامن (Synchronous)، تم اتخاذ قرار بفصل العمليات غير الحرجية ونقلها إلى طبقة معالجة خلفية (Background Processing).

هذا يعني أننا سنقسم عملية ال checkout فعلياً ل عمليتين:

- العملية الأولى هي إنشاء ال order وتتم بشكل فوري
- العملية الثانية هي توليد الفاتورة PDF ويتم تأجيلها وتشغيلها بشكل مستقل.

من نظرة معمارية:

تم اعتماد المكونات التالية:

Laravel Queue System

تم استخدام نظام ال Queue المدمج في Laravel مع Driver من نوع Database لتخزين المهام.

Jobs Layer

تم إنشاء Jobs مستقلة اسمها في المشروع:

▪ GenerateInvoicePdfJob

بحيث تحتوي كل Job على مسؤولية واحدة فقط (Single Responsibility Principle).

3. Dispatch Mechanism

داخل OrderService يتم إرسال المهام إلى ال Queue

4. إعدادات إعادة المحاولة عند الفشل:

```
public $tries = 3;
```

```
public $backoff = 10;
```

التفسير الهندسي: هذه الأسطر تضمن استقرار النظام في حال حدوث أي خطأ غير متوقع أثناء توليد الفاتورة (مثل توقف مؤقت في السيرفر أو مشكلة في كتابة الملف):

- **\$tries = 3:** يعني أنه في حال فشلت ال Job لأي سبب لن تختفي الفاتورة بل سيقوم النظام بإعادة محاولة تنفيذها 3 مرات تلقائياً
- **\$backoff = 10:** يعني أن النظام سينتظر مهلة 10 ثواني بين كل محاولة فاشلة والمحاولة التي تليها ليعطي السيرفر وقتاً للاستجابة وحل المشكلة المؤقتة

الأثر: رفع موثوقية النظام وضمان عدم ضياع فواتير المستخدمين حتى عند حدوث أعطال مفاجئة في البيئة الخلفية.

5 - فحص وجود الملف مسبقاً وتحديث قاعدة البيانات: (Idempotency & Path Registration)

```
    $filePath = "public/invoices/invoice-{$order->id}.pdf";

    if (Storage::exists($filePath)) {
        Log::info(
            "Invoice already exists for order {$order->id}"
        );
        return;
    }

    Storage::put(
        $filePath,
        $pdf->output()
    );
    Log::info("Invoice PDF generated successfully for order {$order->id}");
    $order->update([
        'invoice_path' => $filePath
    ]);
    $jobExecutionTime = microtime(true) - $jobStartTime;

    Log::info('Invoice generation time', [
        'order_id' => $order->id,
        'time_seconds' => $jobExecutionTime,
    ]);
} catch (Throwable $e) {
    Log::error([
        "Invoice generation failed for order {$this->orderId}",
        [
            'error' => $e->getMessage(),
        ]
    ]);
    throw $e;
} } }
```

التفسير الهندسي: هذا الجزء يقوم بحل مشكلتين هامتين بشكل مبسط:

- **الفكرة الأولى (فحص الملف):** باستخدام `Storage::exists` يتم التحقق أولاً إن كان الملف قد تم إنشاؤه مسبقاً أم لا. إذا كان موجوداً، تتوقف الـ Job فوراً (`return`)؛ وهذا يحمي السيرفر من تكرار توليد نفس ملف الـ PDF واستهلاك الموارد في حال تم إعادة محاولة تشغيل الـ Job بالخطأ.
- **الفكرة الثانية (حفظ المسار):** بعد التأكد من إنشاء الملف وحفظه بنجاح، يتم تحديث حقل الـ `invoice_path` داخل قاعدة البيانات الخاص بالطلب (`$order->update`).

الأثر: منع تكرار العمليات غير الضرورية على السيرفر (تحقيق مبدأ ال Idempotency، وتوفير مرجع دائم داخل قاعدة البيانات يربط الطلب بمكان ملف الفاتورة الفعلي ليسهل جلبها للمستخدم لاحقاً).

تدفق النظام (System Flow)

يمر نظام إنشاء الطلب ومعالجة الدفع وفق تسلسل هندسي واضح يفصل بين التنفيذ الفوري والعمليات غير المتزامنة:

1. يقوم المستخدم بإرسال طلب Checkout.
2. يتم إنشاء Order والتحقق من عناصر السلة.
3. يتم تنفيذ عملية التحقق من الرصيد وخصم المبلغ من المحفظة.
4. يتم تسجيل Transaction وتحديث حالة الدفع داخل DB Transaction .
5. بعد نجاح العملية الأساسية، يتم إرسال Job إلى ال Queue:
6. يتم إنهاء الاستجابة للمستخدم مباشرة دون انتظار.
7. في الخلفية (Background Worker):
 - يتم جلب بيانات الطلب
 - توليد ملف PDF للفاتورة
 - تخزين الفاتورة في ال Storage

أهم أجزاء الكود (Key Code Snippets Explained)

في هذه المرحلة، يمكن التركيز على أكثر الأجزاء تأثيراً في التصميم لأنها تمثل التحول الحقيقي نحو المعالجة غير المتزامنة وفصل المسؤوليات داخل النظام.

عملية ال dispatch لل Job:

```
GenerateInvoicePdfJob::dispatch($order->id);
```

التفسير الهندسي:

هذا السطر يمثل نقطة الانتقال من التنفيذ المتزامن إلى التنفيذ غير المتزامن.

- يتم إرسال مهمة إنشاء الفاتورة إلى ال Queue
- لا يتم انتظار تنفيذها داخل ال request
- يتم تحرير دورة حياة ال HTTP مباشرة بعد checkout

الأثر:

تحويل العمليات الثقيلة إلى background processing دون التأثير على استجابة المستخدم

تحميل البيانات داخل ال Job

```
$order = Order::with([
    'user',
    'orderItems.product'
])->find($this->orderId);
```

التفسير الهندسي:

- استخدام Eager Loading لتقليل عدد الاستعلامات (N+1 Problem)
- تجميع كل البيانات المطلوبة للفاتورة في استعلام واحد
- عزل منطق جلب البيانات داخل ال Job

الأثر:

تحسين الأداء وتقليل الضغط على قاعدة البيانات أثناء توليد PDF

النتائج:

لكي أتمكن من كشف الأداء ومراقبة الوقت الذي تستغرقه العمليات بدقة قمت بإضافة دالة القياس الزمنية microtime(true) واستدعاء دالة التسجيل Log::info في مكانين رئيسيين داخل المشروع:

داخل دالة ال Checkout (OrderService): في بداية الدالة و بعد انتهاء الدالة و اكتمال ال transaction و انشاء الطلب

داخل ملف الطابور الخلفي (GenerateInvoicePdfJob): في بداية دالة handle و بعد ان تم حفظ الملف في storage

و كانت هذه هي النتائج :

الفاتورة رقم 22 تم تطبيقها بدون مزامنة

الفاتورة رقم 23 تم تطبيقها مع مبدأ المزامنة

```
[2026-05-16 10:54:25] local.INFO: Checkout execution time {"time_seconds":0.041069984436035156}
[2026-05-16 10:54:25] local.INFO: Invoice PDF generated successfully for order 22
[2026-05-16 10:54:25] local.INFO: Invoice generation time {"order_id":22,"time_seconds":0.2512531280517578}
[2026-05-16 10:55:13] local.INFO: Checkout execution time {"time_seconds":0.02387690544128418}
[2026-05-16 10:55:13] local.INFO: Invoice PDF generated successfully for order 23
[2026-05-16 10:55:13] local.INFO: Invoice generation time {"order_id":23,"time_seconds":0.05103111267089844}
```

الأسطر الثلاث الأولى هي تجربة النظام مع مزامنة و الثلاث الأخيرة تمثل تجربة النظام مع تطبيق اللامزامنة

في التجربة الأولى : يظهر أن عملية checkout أخذت 0.041 ثانية

في التجربة الثانية : عملية checkout أخذت 0.023 ثانية لأن ال Service قامت برمي المهمة إلى ال Queue وتحرير طلب المستخدم

من 0.041 ثانية إلى 0.023 ثانية (تحسن بنسبة %43.9)

في التجربة الأولى : السطر 2 و 3 أظهر أنه تم توليد الملف بنجاح و استغرق وقت 0.251 ثانية

في التجربة الثانية : السطر 2 و 3 أظهر أنه تم توليد الملف بنجاح و استغرق وقت 0.051 ثانية

و السبب في تأخر التجربة الأولى أنه تم استدعاء دالة توليد الملف داخل نطاق DB Transaction قبل القيام ب return كان لا يزال ال CPU محجوز و هو يراقب ال Transaction مما أدى ل بطء في نظام الملفات و الكتابة استغرقت 0.251 ثانية بينما في التجربة الثانية تم قذف المهمة لل queue و تحررت قاعدة البيانات و تم توليد الملف ب بيئة معزولة و نظيفة بزممن 0.051 ثانية

الطلب الرابع: معالجة البيانات على دفعات

Batch Processing in Parallel Environments

في هذا الطلب قمنا بالتركيز على معالجة بيانات المبيعات اليومية الخاصة بمنصة التجارة الإلكترونية بطريقة تسمح للنظام بالتعامل مع عدد كبير من الطلبات دون استهلاك كمية كبيرة من الذاكرة أو التأثير على أداء الخادم.

الفكرة الأساسية كانت إنشاء نظام يقوم بتوليد تقرير يومي للمبيعات يحتوي على تفاصيل الطلبات والإحصائيات العامة ومعلومات الأداء، لكن المشكلة الأساسية ظهرت عند التعامل مع عشرات آلاف الطلبات، لأن تحميل جميع البيانات دفعة واحدة داخل الذاكرة يسبب ضغطاً كبيراً على التطبيق وقد يؤدي إلى بطء شديد أو حتى توقف التنفيذ عند ازدياد حجم البيانات.

لهذا السبب قمنا باستخدام أسلوب Batch Processing، حيث يتم تقسيم البيانات إلى دفعات صغيرة ومعالجة كل دفعة بشكل مستقل، بدلاً من تحميل جميع السجلات دفعة واحدة.

آلية العمل

تم تنفيذ المعالجة على دفعات داخل ChunkedSalesProcessor وتحديداً ضمن الدالة process()

حيث يتم جلب الطلبات باستخدام chunkById() بدلاً من get()، والفرق الأساسي أن get() تقوم بتحميل جميع الطلبات الموجودة في قاعدة البيانات داخل الذاكرة مباشرة، بينما تقوم chunkById() بقراءة البيانات تدريجياً على دفعات صغيرة، وفي مشروعنا تم تحديد حجم الدفعة الواحدة بـ 1000 طلب فقط.

بعد قراءة كل دفعة يتم إرسالها إلى الدالة processBatch() والتي تقوم بحساب الإيرادات وتجهيز بيانات الطلبات والإحصائيات الخاصة بكل دفعة بشكل منفصل عن بقية البيانات.

لماذا استخدمنا Batch Processing

استخدام المعالجة التقليدية كان مناسباً فقط مع البيانات الصغيرة، لكن مع وجود عدد كبير من الطلبات أصبح استهلاك الذاكرة مرتفعاً جداً، لأن النظام يحمل جميع السجلات دفعة واحدة قبل بدء المعالجة.

أما باستخدام Batch Processing فقد أصبح النظام يعالج جزءاً صغيراً من البيانات في كل مرة، وهذا خفف الضغط على الذاكرة بشكل واضح وساعد على تحسين استقرار التطبيق أثناء إنشاء التقارير.

كما أن تقسيم البيانات إلى دفعات أعطانا إمكانية مراقبة أداء كل جزء من المعالجة بشكل مستقل، مثل معرفة الزمن الذي استغرقته كل دفعة وكمية الذاكرة التي استهلكتها أثناء التنفيذ.

مقارنة المعالجة التقليدية مع المعالجة على دفعات

قمنا بإنشاء خدمة باسم DailySalesProcessingService بحيث تسمح للنظام بالعمل بعدة أوضاع، إما باستخدام المعالجة التقليدية أو باستخدام Batch Processing أو تشغيل الطريقتين معاً للمقارنة بين النتائج.

في الوضع التقليدي يتم استخدام NormalSalesProcessor

أما في وضع Batch Processing فيتم استخدام ChunkedSalesProcessor

ثم يتم حساب الفروقات بين الطريقتين من حيث سرعة التنفيذ واستهلاك الذاكرة.

قياس الأداء واستهلاك الذاكرة

لإظهار الفرق الحقيقي بين الطريقتين قمنا بقياس الأداء أثناء التنفيذ باستخدام:

```
memory_get_usage()
memory_get_peak_usage()
microtime(true)
```

حيث يتم تتبع استهلاك الذاكرة منذ بداية المعالجة وحتى نهايتها، بالإضافة إلى تسجيل أعلى قيمة وصلت إليها الذاكرة أثناء التنفيذ.

كما تم تسجيل معلومات تفصيلية لكل دفعة داخل `batches_metrics` بحيث يحتوي كل `Batch` على معلومات مثل عدد الطلبات التي تمت معالجتها والزمن الذي استغرقته العملية واستهلاك الذاكرة قبل وبعد التنفيذ.

الإحصائيات داخل التقرير

بعد انتهاء عملية المعالجة يقوم النظام بحساب مجموعة من الإحصائيات الخاصة بالمبيعات، مثل عدد الطلبات المكتملة أو الملغاة ومتوسط قيمة الطلبات والإيرادات الكلية.

تم تنفيذ ذلك داخل الدالة `calculateOrderStats()` ضمن `ProcessDailySalesJob` ثم يتم إرسال جميع النتائج إلى ملف PDF ليتم إنشاء التقرير النهائي بشكل منظم وواضح.

النتائج

قمت بتنفيذ العملية على 50,000 طلب لقياس الكفاءة الفعلية للعملية، وكانت الإحصائيات الأساسية والمالية كالآتي:

Batch Processing Performance Analysis Report

Sales Statistics

Total Cost	Average Order	Completed	Cancelled	Processing	Pending
13,802,068.00	276.04	24864	5113	12454	7569

وبعدها تم تنفيذ العملية بالطريقتين `NormalSalesProcessor` العادية وباستخدام `ChunkedSalesProcessor` بعد استخدام مفهوم الـ `Batch` وكانت النتائج كالآتي:

Executive Summary

Metric	Normal Processing	Batch Processing
Orders Processed	50000	50000
Execution Time	5.5075s	5.1782s
Peak Memory Usage	448.4081 MB	132.5013 MB
Total Memory Increase	319.8458 MB	86.3921 MB
Processing Status	Completed	Completed
Batch Count	N/A	50
Batch Size	N/A	1000

والنتائج التفصيلية:

Memory Consumption Analysis

Metric	Normal Processing	Batch Processing
Start Memory (Real)	127.5564 MB	41.1633 MB
End Memory (Real)	447.4022 MB	127.5554 MB
Peak Memory (Real)	448.4081 MB	132.5013 MB
Memory Increase	319.8458 MB	86.3921 MB
Allocated Memory	458 MB	136 MB
Number of Batches	N/A	50
Average Batch Memory	N/A	1.542 MB
Largest Batch Memory	N/A	2.0284 MB
Smallest Batch Memory	N/A	1.3585 MB

أظهرت النتائج أن استخدام الـ Batch Processing كان أكثر كفاءة عند التعامل مع أعداد كبيرة من الطلبات، حيث انخفض استهلاك الذاكرة بشكل واضح مقارنة بالطريقة التقليدية، كما أصبح تنفيذ التقرير أكثر استقراراً ومرونة.

Performance Analysis

Memory Reduction %: 70.31%
Batch Processing was 5.98% faster than Normal Processing.

كذلك سمحت المعالجة على دفعات بتقسيم العمل إلى أجزاء صغيرة يمكن مراقبة أدائها بسهولة، وهو ما ساعد على تحسين قابلية التوسع وإدارة الموارد داخل النظام.

Performance Insights

Normal Processing loaded all orders into memory at once, causing very high memory usage of 448.4081 MB. Batch Processing kept memory stable by processing orders in chunks, reducing peak memory to 132.5013 MB. This represents a 70.31% reduction in memory usage.

Batch Processing was 5.98% faster than Normal Processing.

System scalability improved significantly using chunked processing, allowing handling of larger datasets without memory exhaustion.

الطلب الخامس: دراسة وتنفيذ نظام توزيع الأحمال (Load Balancing)

المقدمة والهدف من النظام

يهدف هذا القسم إلى استعراض المعمارية التي تم تبنيتها لتجاوز مشكلة "نقطة الفشل الفردية (Single Point of Failure)" في الأنظمة التقليدية، توجيه جميع الطلبات إلى خادم واحد يشكل مخاطرة كبيرة؛ ففي حال تعطل الخادم، يتوقف النظام بالكامل. لحل هذه المشكلة، تم تصميم هيكلية تعتمد على توزيع الطلبات (Load Balancing) عبر عدة خوادم متوازية، مما يضمن استمرارية الخدمة وتوفير بدائل فورية عند حدوث أي خلل.

الدراسة النظرية: استراتيجيات توزيع الأحمال

قبل اختيار الخوارزمية المناسبة للمشروع، تمت دراسة الخيارات المتاحة وتقييمها:

Round Robin

تُعد خوارزمية Round Robin من أبسط طرق توزيع الحمل. فكرتها تعتمد على إرسال الطلبات بالتسلسل إلى الخوادم الموجودة، مثل:

$A \rightarrow B \rightarrow C \rightarrow A \rightarrow B \rightarrow C \dots$

ميزة هذه الطريقة أنها بسيطة وسريعة ولا تحتاج إلى حسابات معقدة. لكن مشكلتها الأساسية أنها تفترض أن جميع الخوادم تمتلك نفس القدرة والمعالجة، وهذا ليس صحيحاً دائماً. لذلك قد يؤدي هذا الأسلوب إلى توزيع غير ذكي عندما تكون قدرات الخوادم مختلفة.

Weighted Round Robin

للتغلب على مشكلة التوزيع المتساوي بين خوادم غير متساوية في القوة، ظهرت خوارزمية **Weighted Round Robin**. الفكرة هنا هي إعطاء كل خادم **وزناً** يتناسب مع قدرته، بحيث يحصل الخادم الأقوى على عدد أكبر من الطلبات، بينما يحصل الخادم الأضعف على عدد أقل.

هذه الطريقة جيدة لأنها تراعي مواصفات الخادم وقدرته على المعالجة، ولا تحتاج إلى مراقبة لحظية معقدة. لكن عيبها أن الأوزان تكون عادةً ثابتة، وبالتالي لا تتكيف بشكل مباشر مع تغيير الحمل الفعلي على الخادم. فمثلاً، قد يكون الخادم قوياً من حيث العتاد، لكنه مشغول بمهام طويلة أو مستهلك للموارد، ومع ذلك تستمر الخوارزمية في إرسال الطلبات إليه لأن وزنه ما زال مرتفعاً.

Least Connections

خوارزمية Least Connections تعتمد على مبدأ مختلف، وهو إرسال الطلب الجديد إلى الخادم الذي يملك أقل عدد من الاتصالات النشطة حالياً. بمعنى آخر، عند وصول كل طلب جديد، يتم اختيار الخادم الأقل انشغالاً في تلك اللحظة.

تعتمد هذه الطريقة على عداد لكل خادم:

- يزداد عند وصول اتصال جديد.
- ينقص عند انتهاء الاتصال.

وتُستخدم هذه الاستراتيجية كثيراً في التطبيقات التي تكون فيها مدة الاتصال مهمة، مثل:

- WebSocket
- بعض تطبيقات AI inference

لكن لهذه الخوارزمية أيضاً محدودية، وهي أنها تفترض أن عدد الاتصالات يعكس الحمل الحقيقي دائماً، وهذا غير دقيق في كل الحالات. فقد يكون خادم لديه 5 اتصالات لكنها طلبات ثقيلة جداً، بينما خادم آخر لديه 10 اتصالات لكنها طلبات خفيفة. في هذه الحالة، الخوارزمية قد تعتبر الخادم الثاني أكثر انشغالاً، رغم أن الأول هو فعلياً الأكثر ضغطاً.

Weighted Least Connections

لتحسين الخوارزمية السابقة، ظهرت **Weighted Least Connections**. هذه الخوارزمية لا تنظر فقط إلى عدد الاتصالات الحالية، بل تأخذ أيضاً في الاعتبار قوة الخادم من خلال وزنه.

بالتالي، بدل أن نسأل:
من لديه أقل عدد اتصالات؟

نطرح سؤالاً أدق:
من هو الأقل ضغطاً مقارنة بقدراته؟

ويتم ذلك عبر حساب Load Score لكل خادم، عادةً من خلال:

$$\text{Load Score} = \text{عدد الاتصالات الحالية} \div \text{وزن الخادم}$$

ثم يتم اختيار الخادم الذي يملك أقل Load Score عند وصول كل طلب جديد.

هذه الخوارزمية أفضل من Least Connections لأنها تراعي اختلاف قدرات الخوادم، وهذا مهم جداً في البيئات الحقيقية لأن الخوادم غالباً ليست متساوية.

هذه الاستراتيجية مختلفة عن الخوارزميات السابقة، لأن هدفها ليس بالضرورة تحقيق أفضل توزيع للحمل، بل ربط نفس المستخدم بنفس الخادم دائماً. وهذا يفيد في حالة Sticky Sessions.

آلية العمل بسيطة:

- يتم أخذ عنوان IP الخاص بالمستخدم.
- يتم تطبيق hash عليه.
- ثم يتم حساب:

$\text{hash \% number_of_servers}$

والنتيجة تحدد الخادم الذي سترسل إليه طلبات هذا المستخدم.

تُستخدم هذه الطريقة عندما نريد الحفاظ على جلسة المستخدم على نفس الخادم، لكن استخدامها أصبح أقل في الأنظمة الحديثة، لأن أغلب التطبيقات أصبحت Stateless، أي أن أي خادم يستطيع خدمة أي طلب. وفي هذه الحالة يتم تخزين الجلسات في أماكن مشتركة مثل Redis، بدل الاعتماد على ذاكرة الخادم نفسه.

ومع ذلك، لهذه الطريقة عيوب مهمة:

- عند إضافة خادم جديد، قد تتغير خريطة التوزيع.
- عند تعطل خادم، قد يتغير مسار المستخدمين المرتبطين به.
- قد تؤدي إلى فقدان بعض البيانات إذا كانت الجلسات معتمدة على الخادم نفسه.

التنفيذ العملي: المعمارية التقنية

بناءً على الدراسة النظرية، تم اختيار خوارزمية Least Connections لتطبيقها عملياً باستخدام (Docker) و (NGINX). السبب وراء هذا الاختيار هو قدرة هذه الخوارزمية على حماية الخوادم من تراكم الطلبات الثقيلة (مثل عمليات الدفع أو تحديث المخزون) تلقائياً وأنه موارد الخوادم المتعددة متساوية تقريباً فمن غير المنطقي إضافة خوارزمية فيها اوزان.

لتطبيق هذه الأفكار عملياً، تم استخدام:

- NGINX ك Load Balancer
- Docker لتشغيل أكثر من نسخة من التطبيق
- MySQL كقاعدة بيانات مشتركة
- Redis لتخزين البيانات المشتركة والجلسات عند الحاجة

الهدف من هذا التطبيق العملي هو محاكاة بيئة فيها أكثر من خادم تطبيق، مع وجود خادم أمامي يقوم بتوزيع الطلبات بينها، ثم اختبار سلوك النظام بعد ذلك.

شرح الملفات التي تم اعدادها

Dockerfile

تم بناء صورة التطبيق باستخدام:

```
FROM php:8.4-fpm
```

وهذا يعني أن التطبيق سيعمل داخل حاوية PHP-FPM، وهو الخيار المناسب لتشغيل تطبيقات Laravel أو التطبيقات المبنية على PHP مع NGINX.

سبب اختيار هذه الحزمة:

- php-fpm مناسب للتكامل مع NGINX
- يوفر أداء أفضل من تشغيل PHP بطريقة تقليدية داخل خادم ويب واحد

تفعيل إضافات PHP وتثبيت الحزم المطلوبة

إضافة Redis

```
RUN pecl install redis && docker-php-ext-enable redis
```

تمت إضافة Redis لأن النظام يحتاج إلى تخزين بيانات مشتركة أو جلسات بشكل مركزي بدل ربطها بخادم واحد.

نسخ Composer

```
COPY --from=composer:latest /usr/bin/composer /usr/bin/composer
```

تم استخدام Composer لتثبيت مكتبات Laravel وإدارة الاعتمادات الخاصة بالمشروع.

نسخ ملفات المشروع وتشغيل Composer

```
WORKDIR /var/www
```

```
. COPY .
```

```
RUN composer install --no-dev --optimize-autoloader
```


السبب في ذلك هو تجهيز المشروع داخل الحاوية وتثبيت الحزم اللازمة له، مع تحسين التحميل عبر optimize-autoloader.

تشغيل PHP-FPM

EXPOSE 9000

CMD ["php-fpm"]

تم فتح المنفذ 9000 لأن NGINX سيتواصل مع PHP-FPM من خلاله.

docker-compose.yml

تم استخدام Docker Compose لتشغيل النظام بالكامل كبيئة متعددة الحاويات.

مكونات النظام:

- loadbalancer: خادم NGINX الأمامي
- app1, app2, app3: ثلاث نسخ من التطبيق
- app1-web, app2-web, app3-web: خوادم NGINX خاصة بكل نسخة من التطبيق
- mysql: قاعدة البيانات
- redis: التخزين المشترك

لماذا تم تقسيم التطبيق إلى أكثر من حاوية؟

هذا التقسيم يحاكي بيئة حقيقية فيها أكثر من node، بحيث يمكن اختبار موازنة الحمل عملياً بدل الاكتفاء بالشرح النظري فقط.

شبكة مشتركة

تم ربط جميع الخدمات على شبكة واحدة:

networks:

ecommerce-network:

driver: bridge

الهدف من ذلك هو تمكين الحاويات من التواصل الداخلي فيما بينها باستخدام أسماء الخدمات مثل app1 و mysql و redis.

تم تشغيل Redis وMySQL كخدمات مستقلة، لأن هذا أفضل من ربط البيانات أو الجلسات بحاوية تطبيق واحدة.

هذا يحقق مبدأ مهماً في الأنظمة الحديثة، وهو:

- فصل البيانات عن التطبيق
- جعل التطبيق أقرب إلى Stateless
- تسهيل التوسع لاحقاً

إعدادات NGINX الخاصة بكل تطبيق

تم إنشاء إعدادات مستقلة لكل Node داخل النظام، بحيث يمتلك كل خادم إعدادات NGINX خاصة به تتولى استقبال الطلبات وتوجيهها إلى تطبيق Laravel العامل عبر PHP-FPM.

كان الهدف من هذا التقسيم هو محاكاة بيئة حقيقية تحتوي على عدة خوادم مستقلة، بحيث يستطيع الـ Load Balancer توزيع الطلبات بينها بسهولة.

تضمنت هذه الإعدادات عدة نقاط مهمة:

- تحديد المجلد العام الخاص بتطبيق Laravel ليكون نقطة الدخول الأساسية للتطبيق.
- تمرير الطلبات الديناميكية الخاصة بـ PHP إلى خدمة PHP-FPM الموجودة داخل نفس الحاوية.
- دعم Laravel Routing بشكل صحيح، بحيث يتم توجيه الطلبات إلى Routes الخاصة بالتطبيق حتى في حال عدم وجود ملف فعلي للمسار المطلوب.
- تحسين استقرار الاتصال والأداء عبر إعدادات خاصة بـ Keep Alive وتقليل إعادة إنشاء الاتصالات بشكل متكرر.
- رفع الحد الأقصى لحجم الطلبات والملفات المرفوعة ليتناسب مع متطلبات المشروع.
- إضافة Header مخصص لكل Node بهدف معرفة أي خادم قام بمعالجة الطلب أثناء الاختبار، مما ساعد على التأكد من أن الـ Load Balancer يوزع الطلبات فعلياً بين الخوادم المختلفة.

وبذلك أصبحت كل نسخة من التطبيق تعمل كخادم مستقل يمكن لموازن الحمل التعامل معه وتوجيه الطلبات إليه بشكل مباشر.

إعداد PHP-FPM

تم تخصيص إعدادات PHP-FPM بما يتناسب مع طبيعة النظام وعدد الطلبات المتوقع التعامل معها داخل بيئة متعددة الخوادم.

كان الهدف الأساسي من هذه الإعدادات هو تحسين أداء معالجة طلبات PHP، وتنظيم استهلاك الموارد، وتحقيق استقرار أكبر أثناء الضغط.

تم تشغيل الخدمة على المنفذ المخصص للتواصل الداخلي بين NGINX و PHP-FPM، بحيث يقوم NGINX بتمرير طلبات PHP إلى محرك المعالجة مباشرة.

كما تم استخدام نمط الإدارة الديناميكية للعمليات (Dynamic Process Management)، والذي يسمح بإنشاء وإدارة عدد العمال (Workers) تلقائياً حسب حجم الضغط الحالي على التطبيق. وهذا يساعد على تحقيق توازن جيد بين الأداء واستهلاك الموارد، لأن النظام لا يقوم بتشغيل عدد كبير من العمليات بشكل دائم دون حاجة.

كذلك تم تحديد حدود لعدد العمليات والاتصالات المتزامنة بهدف:

- منع استهلاك الذاكرة بشكل مفرط.
- تحسين استجابة التطبيق تحت الضغط.
- تقليل احتمالية توقف الخدمة بسبب زيادة الحمل.

وتم أيضاً تحديد حد أقصى لعدد الطلبات التي يعالجها كل Worker قبل إعادة إنشائه، وذلك لتقليل مشاكل استهلاك الذاكرة التراكمي وتحسين استقرار التطبيق على المدى الطويل.

إعداد Load Balancer باستخدام NGINX

تم إعداد NGINX ليعمل كموازن حمل (Load Balancer) يقوم باستقبال الطلبات القادمة إلى النظام وتوزيعها على خوادم التطبيق المختلفة الموجودة في الخلفية.

كما تم إعداد الموازن للتعامل مع حالات الفشل، بحيث عند تعطل أحد الخوادم أو فشله عدة مرات متتالية، يتوقف النظام مؤقتاً عن إرسال الطلبات إليه حتى يعود للعمل مجدداً، مما يساعد على تحسين استقرار الخدمة.

كذلك تم إضافة Headers مخصصة أثناء الاختبار لمعرفة الخادم الذي قام بمعالجة الطلب فعلياً، والتأكد من أن عملية توزيع الطلبات تعمل بالشكل الصحيح بين ال Nodes المختلفة.

وتم أيضاً تمرير معلومات العميل الأصلية مثل عنوان ال IP الحقيقي، وذلك لأغراض التسجيل والتتبع والتحليل.

النتيجة العملية

بعد بناء الصور وتشغيل الحاويات، تم تشغيل:

- 3 حاويات للتطبيق

- 3 حاويات NGINX خاصة بكل تطبيق
- موازن حمل رئيسي
- MySQL
- Redis

وهذا يثبت أن البيئة متعددة الخوادم تعمل بشكل صحيح، وأن الطلبات يمكن توجيهها إلى أكثر من Node بدل الاعتماد على خادم واحد فقط.

كما أنه تم عمل Horizontal Scaling بإنشاء ثلاث نسخ مستقلة (app1, app2, app3) تعمل بالتوازي وتوزيع الجهد بينهم حيث كل عقدة لديها حاوية PHP-FPM و Nginx خاصة بها

لوحة التحكم والمراقبة (Dashboard)

للإثبات كفاءة النظام، تم تطوير لوحة تحكم متقدمة باستخدام (JavaScript) للتفاعل الحي مع البيئة:

- **المراقبة الحية (Monitor):** تقوم اللوحة بإرسال طلبات متزامنة وتسجيل زمن الاستجابة، وحالة النجاح والفشل، واسم الخادم الذي قام بالمعالجة. هذا يثبت أن الطلبات تتوزع فعلياً على الخوادم الثلاثة.
- **اختبار الضغط (Stress Test):** تم برمجة وحدة لمحاكاة دخول مئات المستخدمين الوهميين معاً. يمكن اختبار النظام تحت أحمال خفيفة أو ثقيلة لمعرفة أقصى طاقة استيعابية (Requests Per Second) للنظام قبل أن يبدأ بالانهيار.
- **مقارنة الخوارزميات (Algorithm Comparison):** تقوم اللوحة باختبار الخوارزمية الحالية (Least Connections) الفعالة في النظام، ثم تقوم بمحاكاة برمجة لعمل خوارزمية (Round Robin) على نفس الطلبات. النتائج المعروضة توضح تفوق Least Connections في تقليل متوسط زمن الاستجابة (Average Latency).
- **محاكاة الفشل (Failure Simulation/Chaos Testing):** وهي أهم ميزة في اللوحة. تتيح إرسال أمر فعلي عبر (API) لإيقاف تشغيل حاوية (Node) معينة داخل (Docker) عند الإيقاف، تقوم اللوحة بمراقبة كيف يتعامل (NGINX) مع هذا السقوط وكيف يعيد توجيه جميع الطلبات تلقائياً إلى الخوادم السليمة المتبقية دون أن يشعر المستخدم بأي توقف بالخدمة.

النتائج عند وجود Node واحدة فقط (أي وجود الـ load balancer وعدمه واحد)



النتائج عند تشغيل 3 عقد (3 سيرفرات) واستخدام load balancer لتوزيع الحمل:



The background features abstract, organic shapes in teal, green, and dark teal. A small white curly brace is located in the top left corner. The text "Thank You" is centered in a black, handwritten-style font with a subtle drop shadow.

Thank You